

Midnight Builder Challenge

AI Prompts — New Moon to Full · One prompt per level

Paste each prompt into Claude or Cursor at the start of that level. The AI handles all code, file structure, and README generation. The only things you do manually are commits and — at Level 5 — collecting user feedback.

⚠ DO THIS MANUALLY

- Commits — make them yourself after each milestone, with meaningful messages
- User feedback (Level 5 only) — go get 50 real people to test your Preprod link

Contents

- 🌑 L1 · New Moon — Setup & First Contract · page 2
- 🌒 L2 · Waxing Crescent — Frontend Integration · page 3
- 🌓 L3 · First Quarter — Production-Grade dApp · page 4
- 🌔 L4 · Waxing Gibbous — MVP Goes Live · page 5
- 🌕 L5 · Full Moon — Users & Feedback · page 6
- 🌕 L6 · Supermoon — Mainnet Launch · page 7

🌑 NEW MOON · Level 1 — Setup & First Contract

🌑 Level 1 — Setup & First Contract

No prize — entry level. Complete to unlock the prize track from Level 2 onward.

Prompt

Copy and paste this entire prompt into Claude or Cursor:

```
You are helping me complete Level 1 of the Midnight Builder Challenge on Rise In.
My project folder is: [PASTE YOUR PROJECT PATH HERE]

=====
MIDNIGHT DOCS MCP — ADD THIS FIRST
=====

Before starting, make sure the Midnight documentation MCP is connected.
Run this command in your terminal:
  claude mcp add --transport http midnight-docs https://midnight.mcp.kapa.ai
Or access the docs directly at: https://midnight.mcp.kapa.ai
This gives you live Midnight documentation inside every AI response.
```

Do the following steps in order. Do not skip any step.

STEP 1 - TOOLCHAIN SETUP

- Verify Node.js v22 is installed. If not, tell me to install it before continuing.
- Verify Docker is running.
- Install the Compact compiler:
npm install -g @midnight-ntwrk/compact-compiler
- Pull the proof server:
docker pull midnightnetwork/proof-server
- Run the proof server:
docker run -p 6300:6300 midnightnetwork/proof-server
- Verify: compact --version
- Confirm you see a version number before continuing.

STEP 2 - MCP SETUP

- Open my Claude Desktop config:
~/Library/Application Support/Claude/claude_desktop_config.json
- Add the Midnight MCP entry:
{ "midnight": { "command": "npx", "args": ["-y", "@midnight-ntwrk/mcp-server"] } }
- If I am on Cursor: Settings → MCP → Add Server →
npx -y @midnight-ntwrk/mcp-server

STEP 3 - HELLO WORLD DEPLOY

- Scaffold:
npx -y create-mn-app mn-demo --template hello-world --use-npm
cd mn-demo
- Deploy to preview:
NODE_OPTIONS="--max-old-space-size=12288" npm run deploy -- --network preview
- STOP when the wallet address prints. Tell me to fund it at the preview faucet.
- Wait for me to confirm funding before continuing.
- After deploy completes: npm run network preview
- Print the deployed contract address clearly.

STEP 4 - PROJECT FILE STRUCTURE

Create the following folder structure in my project root:

```
my-project/
├── contracts/
│   └── counter.compact      ← my Compact contract
├── managed/                ← auto-generated by compact compile
├── src/                    ← frontend (added in Level 2)
├── tests/
│   └── counter.test.ts     ← test file
├── .github/
│   └── workflows/          ← CI/CD added in Level 3
```

```
├── README.md          ← detailed README (see Step 6)
└── package.json
```

STEP 5 - WRITE AND COMPILE THE CONTRACT

- Write contracts/counter.compact with:
 - a) At least one piece of public ledger state
 - b) At least one private witness as a circuit input
 - c) At least one disclose() used deliberately
 - d) A comment block at the top explaining what is public vs private
- Compile: compact compile
- Confirm the managed/ directory was created with circuits and keys.
- Write tests/counter.test.ts with at least 3 passing tests covering:
 - Circuit logic
 - State transitions
 - That private inputs are never exposed
- Run tests and confirm all pass.
- Deploy my contract (not the hello-world) to Preview or Preprod.
- Print the deployed contract address clearly.

STEP 6 - README.md (MANDATORY - DO NOT SKIP)

Create a detailed README.md in the project root with ALL of these sections:

```
# [Project Name]
> One-line description of what this contract does.

## Contract Address
| Network | Address |
|-----|-----|
| Preview | [PASTE ADDRESS AFTER DEPLOY] |
| Preprod | [PASTE ADDRESS AFTER DEPLOY] |

(This section is MANDATORY. Leave placeholders if not deployed yet.)

## What This Does
Plain English explanation of the contract's purpose.

## Privacy Model
- What is PUBLIC (on-chain, visible to anyone):
- What is PRIVATE (private witness, never on-chain):
- What the user PROVES without revealing:

## Tech Stack
- Midnight network, Compact language, Node.js v22, Docker

## Prerequisites
List everything needed to run locally.

## Setup
Step-by-step commands to clone, install, and run.

## Run Tests
```

```
Command to run the test suite.
```

```
## Initial Idea
```

```
[LEAVE PLACEHOLDER — I will fill this in manually]
```

```
## Screenshots
```

```
[LEAVE PLACEHOLDER — I will add compile output and contract address screenshots]
```

STEP 7 — FINAL CHECKLIST

When all steps are done, print a checklist showing:

✓ or ✗ for each requirement below:

- [] Contract compiles with compact compile
- [] managed/ directory present
- [] 3+ tests passing
- [] Contract deployed to Preview or Preprod
- [] Contract address visible in README.md
- [] README has all required sections
- [] File structure matches the spec

Then remind me to fill in the Initial Idea, add screenshots,
and make at least 5 meaningful commits before submitting on Rise In.

⚠ DO THIS MANUALLY

- Fund the preview faucet wallet when the terminal pauses and tells you to
- Fill in the Initial Idea section in README.md yourself
- Paste the deployed contract address into the README Contract Address table
- Take screenshots: compile output + deployed address, add to README
- Make at least 5 meaningful commits with clear messages
- Submit your public GitHub repo on Rise In

🌙 WAXING CRESCENT · Level 2 — Frontend Integration

🌙 Level 2 — Frontend Integration

PRIZE POOL

🌙 Waxing Crescent — Level 2 · 60 winners × \$10 each = \$600 total

Prompt

Copy and paste this entire prompt into Claude or Cursor:

You are helping me complete Level 2 of the Midnight Builder Challenge on Rise In.
My repo from Level 1 is at: [PASTE REPO PATH]
My Preprod contract address is: [PASTE CONTRACT ADDRESS]

MIDNIGHT DOCS MCP — ADD THIS FIRST

Before starting, make sure the Midnight documentation MCP is connected.

Run this command in your terminal:

```
claude mcp add --transport http midnight-docs https://midnight.mcp.kapa.ai
```

Or access the docs directly at: <https://midnight.mcp.kapa.ai>

This gives you live Midnight documentation inside every AI response.

Do the following steps in order. Do not skip any step.

STEP 1 — FILE STRUCTURE

Extend the Level 1 structure by adding the frontend:

```
my-project/
├── contracts/
│   └── counter.compact
├── managed/
├── src/
│   ├── components/
│   │   ├── WalletConnect.tsx    ← wallet connect/disconnect UI
│   │   └── CircuitCall.tsx      ← circuit call button + result display
│   ├── hooks/
│   │   └── useMidnight.ts        ← Midnight.js SDK hook
│   ├── App.tsx
│   └── main.tsx
├── tests/
├── public/
├── .github/
├── README.md
├── package.json
└── vite.config.ts (or next.config.js)
```

STEP 2 — FRONTEND SETUP

- Scaffold a React + Vite project (or Next.js) inside the repo.
- Install Midnight.js SDK and DApp connector:

```
npm install @midnight-ntwrk/midnight-js-network-provider
npm install @midnight-ntwrk/dapp-connector-api
```
- Confirm the project builds with no errors.

STEP 3 — WALLET CONNECTION

- Build WalletConnect.tsx:
 - Connect button → triggers Lace wallet connection
 - Disconnect button → clears wallet state
 - Shows connected wallet address on screen when connected

- Shows clear disconnected state when not connected
- Handles errors: wallet not installed, user rejected, network mismatch

STEP 4 - CIRCUIT CALL

- Build CircuitCall.tsx:
 - Button that calls a circuit from my Preprod contract
 - Proof is generated locally in the browser
 - Result is submitted on-chain
 - Loading state shown during proof generation
 - Transaction result displayed after submission
 - Private inputs MUST NEVER appear in the UI
 - Add a label: 'Proved without revealing your input'

STEP 5 - DEPLOY FRONTEND

- Add vercel.json or netlify.toml to the repo.
- Give me the exact CLI commands to deploy.
- The live URL must connect to my Preprod contract address.

STEP 6 - README.md (MANDATORY - DO NOT SKIP)

Update README.md to include ALL of these sections:

```
# [Project Name]
> One-line description.
```

```
## Live Demo
[PASTE LIVE URL AFTER DEPLOYING FRONTEND]
```

```
## Contract Address
| Network | Address |
|-----|-----|
| Preprod | [CONTRACT ADDRESS FROM LEVEL 1] |
```

(Contract address is MANDATORY. Do not leave this blank.)

```
## What This Does
Plain English description of the dApp.
```

```
## Privacy Model
- What is PUBLIC:
- What is PRIVATE:
- What the user PROVES without revealing:
```

```
## Privacy Claim
Specific statement: what an on-chain observer sees vs cannot see.
```

```
## Tech Stack
Midnight network, Compact, Midnight.js SDK, React/Vite, Lace wallet
```

```
## Prerequisites
```

```
- Lace wallet installed
- Node.js v22

## Run Locally
Step-by-step clone → install → run commands.

## Demo Video
[PLACEHOLDER — I will add the link after recording]
```

STEP 7 — DEMO VIDEO CHECKLIST

Tell me exactly what to record in the demo video (under 2 minutes):

1. Connect Lace wallet — show the address appear on screen
2. Call the circuit — show the loading state during proof generation
3. Show the on-chain result after submission
4. Point out that the private input was never shown

STEP 8 — FINAL CHECKLIST

Print ✓ or ✗ for each requirement:

- [] Lace wallet connect and disconnect working
- [] Circuit called from frontend, proof generated locally
- [] Private input never shown in UI
- [] Contract address in README.md (MANDATORY)
- [] Live demo link in README.md
- [] Privacy Claim section in README.md
- [] File structure matches spec

Then remind me to deploy, record the demo video, and commit.

⚠ DO THIS MANUALLY

- Deploy to Vercel or Netlify and paste the live URL into README.md
- Paste your contract address into the README Contract Address table — mandatory
- Record the demo video following the AI's checklist
- Make at least 8 meaningful commits with clear messages
- Submit your GitHub repo + live link on Rise In

🌑 FIRST QUARTER · Level 3 — Production-Grade dApp

🌑 Level 3 — Production-Grade dApp

PRIZE POOL

🌑 First Quarter — Level 3 · 55 winners × \$30 each = \$1,650 total

Prompt

Copy and paste this entire prompt into Claude or Cursor:

```
You are helping me complete Level 3 of the Midnight Builder Challenge on Rise In.  
My repo from Level 2 is at: [PASTE REPO PATH]  
My Preprod contract address is: [PASTE CONTRACT ADDRESS]
```

```
=====
```

MIDNIGHT DOCS MCP — ADD THIS FIRST

```
=====
```

Before starting, make sure the Midnight documentation MCP is connected.
Run this command in your terminal:
 `claude mcp add --transport http midnight-docs https://midnight.mcp.kapa.ai`
Or access the docs directly at: <https://midnight.mcp.kapa.ai>
This gives you live Midnight documentation inside every AI response.

Do the following steps in order. Do not skip any step.

```
=====
```

STEP 1 — FILE STRUCTURE CHECK

```
=====
```

Verify and enforce this structure. Create any missing files/folders:

```
my-project/  
├── contracts/  
│   └── counter.compact  
├── managed/                               ← must exist after compile  
├── src/  
│   ├── components/  
│   ├── hooks/  
│   ├── App.tsx  
│   └── main.tsx  
├── tests/  
│   └── counter.test.ts  
├── .github/  
│   └── workflows/  
│       └── ci.yml                         ← create this in Step 2  
├── PROPOSAL.md                           ← create this in Step 4  
├── README.md  
└── package.json
```

```
=====
```

STEP 2 — TESTS (MINIMUM 3)

```
=====
```

- Review or write tests/counter.test.ts.
 - Must have at least 3 tests covering:
 - a) Circuit logic — does the circuit compute correctly?
 - b) State transitions — does ledger state update as expected?
 - c) Privacy — private input is never exposed in any output
 - Run tests: confirm all pass.
 - Show me the test output.
- ```
=====
```

### STEP 3 - CI/CD PIPELINE

- 
- Create `.github/workflows/ci.yml`
  - Triggers on: push to main and pull\_request
  - Steps:
    1. Checkout code
    2. Install Node.js v22
    3. npm install
    4. compact compile
    5. Run test suite
  - Add the CI status badge to the top of README.md immediately below the title.
- 

### STEP 4 - POLISH THE DAPP

Review the frontend and fix:

- All error states handled with clear user messages
- Loading spinner or indicator during proof generation
- Privacy behavior clearly labeled in the UI
- Mobile-responsive layout
- No console errors in production build

Run: `npm run build` - confirm zero errors.

---

### STEP 5 - PROPOSAL.md

Create PROPOSAL.md in the root with this exact structure:

```
Product Proposal

What is the product, and who uses it?
[I WILL FILL THIS IN]

Why Midnight specifically?
[I WILL FILL THIS IN - what does Midnight do that a transparent
chain could not do well for this product?]

Data Model
Data Point	Type	Disclosed To
[example]	Public ledger	Everyone
[example]	Private witness	No one
[I WILL FILL IN THE ROWS]

Mainnet Feasibility
[I WILL FILL THIS IN - is this realistic to reach Mainnet by Level 6?]
```

Leave all placeholders - I will fill in my answers manually.

---

### STEP 6 - README.md (MANDATORY - DO NOT SKIP)

Update README.md to include ALL of these sections in this order:

```
[Project Name]
```

```

! [CI] (badge-url)
> One-line description.

Live Demo
[Live URL]

Contract Address ← MANDATORY
Network	Address
Preprod	[CONTRACT ADDRESS - REQUIRED]

What This Does

Privacy Model
- PUBLIC:
- PRIVATE:
- PROVED without revealing:

Privacy Claim
What an on-chain observer sees vs cannot see.

Tech Stack

Prerequisites

Setup & Run Locally
(step-by-step commands)

Run Tests
...
npm test
...

CI/CD
Explain what the pipeline does.

Product Proposal
See PROPOSAL.md

```

---

## STEP 7 – DEMO VIDEO CHECKLIST

---

Tell me what to show in the 1-minute demo video:

1. Full dApp flow: wallet connect → circuit call → result
2. Terminal showing test output (3+ passing)
3. README showing CI badge as green

---

## STEP 8 – FINAL CHECKLIST

---

Print ✓ or ✗ for each requirement:

- [ ] 3+ tests passing
- [ ] CI/CD pipeline running on push
- [ ] CI badge in README.md
- [ ] Contract address in README.md (MANDATORY)

```
[] Privacy Model section in README.md
[] PROPOSAL.md created with correct structure
[] dApp builds with zero errors
[] File structure matches spec
Then remind me to fill in PROPOSAL.md and make 10 commits.
```

#### ⚠ DO THIS MANUALLY

- Fill in all sections of PROPOSAL.md yourself — this is your product idea
- Ensure your Preprod contract address is in the README — this is mandatory
- Record the 1-minute demo video following the AI's checklist
- Make at least 10 meaningful commits with clear messages
- Submit on Rise In and await idea approval before starting Level 4

### 🌑 WAXING GIBBOUS · Level 4 — MVP Goes Live

## 🌑 Level 4 — MVP Goes Live

#### PRIZE POOL

🌑 Waxing Gibbous — Level 4 · 25 winners × \$60 each = \$1,500 total

Only start Level 4 after your product proposal from Level 3 has been approved at The Turn.

### Prompt

Copy and paste this entire prompt into Claude or Cursor:

```
You are helping me complete Level 4 of the Midnight Builder Challenge on Rise In.
My approved product idea: [PASTE APPROVED IDEA]
New repo path: [PASTE PATH]

=====
MIDNIGHT DOCS MCP — ADD THIS FIRST
=====

Before starting, make sure the Midnight documentation MCP is connected.
Run this command in your terminal:
 claude mcp add --transport http midnight-docs https://midnight.mcp.kapa.ai
Or access the docs directly at: https://midnight.mcp.kapa.ai
This gives you live Midnight documentation inside every AI response.

Do the following steps in order. Do not skip any step.

=====
```

## STEP 1 – FILE STRUCTURE (SET UP FIRST)

Create this complete folder structure before writing any code:

```
my-product/
├── contracts/
│ └── [product-name].compact ← main contract
├── managed/ ← auto-generated
├── src/
│ ├── components/
│ │ ├── WalletConnect.tsx
│ │ ├── [CoreFeature].tsx ← main privacy feature UI
│ │ └── Layout.tsx
│ ├── hooks/
│ │ └── useMidnight.ts
│ ├── utils/
│ │ └── contract.ts ← contract interaction helpers
│ ├── App.tsx
│ └── main.tsx
├── tests/
│ └── [product-name].test.ts
├── .github/
│ └── workflows/
│ └── ci.yml
├── docs/
│ └── USAGE.md ← how to use the product
├── README.md
├── PROPOSAL.md ← copy from Level 3
└── package.json
```

## STEP 2 – CONTRACT (PRIVACY CORE FIRST)

- Write the Compact contract for my approved product.
- Build the privacy logic before any UI.
- The contract must:
  - a) Have public ledger state for what must be verifiable on-chain
  - b) Use private witnesses for all sensitive inputs
  - c) Use disclose() only where deliberately needed
  - d) Have a comment block at the top explaining the privacy model
- Compile: compact compile
- Write tests – minimum 3 passing.
- Run tests and confirm all pass.

## STEP 3 – FRONTEND

- Build the frontend wired to the contract.
- Privacy behavior must be the central feature, not a footnote.
- Must include: wallet connect, circuit calls, loading states, error states.
- Build check: npm run build – zero errors required.

## STEP 4 – CI/CD

```
- Create .github/workflows/ci.yml
- On push to main: install → compact compile → run tests
- Add CI badge to README.md
```

---

#### STEP 5 – DEPLOY TO PREPROD

---

```
- Give me the exact deploy command for my contract to Preprod.
- STOP and wait for me to run the deploy and paste back the contract address.
- After I paste the address, update README.md immediately.
```

---

#### STEP 6 – docs/USAGE.md

---

Create docs/USAGE.md with:

```
How to Use [Product Name]
What You Need
Step-by-Step Guide
 (numbered steps, plain English, non-technical user friendly)
What Gets Proved (and What Stays Private)
Troubleshooting
```

---

#### STEP 7 – README.md (MANDATORY – DO NOT SKIP)

---

Write a complete README.md with ALL sections in this order:

```
[Product Name]
![CI](badge-url)
> Tagline: what it does in one sentence.

Live Demo
[Preprod demo URL – I will paste after deploying frontend]

Contract Address ← MANDATORY – submission is invalid without this
Network	Address
Preprod	[ADDRESS – I will paste after deploy]

What This Product Does
(2-3 paragraphs: what problem, who uses it, why Midnight)

Privacy Model
- What is PUBLIC (on-chain, anyone can see):
- What is PRIVATE (private witness, never on-chain):
- What the user PROVES without revealing:

Tech Stack

Prerequisites
(Lace wallet, Node v22, Docker)

Setup & Run Locally
(numbered step-by-step commands)
```

```
Run Tests

CI/CD

Usage Guide
See docs/USAGE.md

Product X Profile
[PLACEHOLDER — I will add after creating the account]
```

---

#### STEP 8 — X PROFILE LAUNCH POSTS

---

Write 3 ready-to-post tweets for my product X account:

- Tweet 1: what the product is and why it needs Midnight
- Tweet 2: a technical insight about the privacy model
- Tweet 3: call to try the Preprod demo (include the link)

---

#### STEP 9 — FINAL CHECKLIST

---

Print ✓ or ✗ for each requirement:

- ☐ Contract compiled and tests passing
- ☐ Contract deployed to Preprod
- ☐ Contract address in README.md (MANDATORY)
- ☐ Live Preprod demo link in README.md
- ☐ CI/CD running and badge in README.md
- ☐ docs/USAGE.md created
- ☐ File structure matches spec
- ☐ npm run build passes with zero errors

Then remind me to deploy, create the X account, post the tweets, add the X link to README, record the demo video, and commit.

#### ⚠ DO THIS MANUALLY

- Run the Preprod deploy and paste the contract address back — then let the AI update the README
- Paste the contract address into the README Contract Address table — mandatory
- Deploy frontend to Vercel/Netlify and add the live URL to README
- Create the product X account, post the 3 tweets, add the profile link to README
- Record the MVP demo video
- Make at least 15 meaningful commits with clear messages
- Submit your GitHub repo on Rise In

## 🌕 Level 5 — Users & Feedback

### PRIZE POOL

🌕 Full Moon — Level 5 · 20 winners × \$100 each = \$2,000 total

### Prompt

Copy and paste this entire prompt into Claude or Cursor:

```
You are helping me complete Level 5 of the Midnight Builder Challenge on Rise In.
My repo from Level 4 is at: [PASTE REPO PATH]
My Preprod contract address is: [PASTE CONTRACT ADDRESS]
My live Preprod demo link is: [PASTE LINK]
```

```
=====
```

MIDNIGHT DOCS MCP — ADD THIS FIRST

```
=====
```

Before starting, make sure the Midnight documentation MCP is connected.  
Run this command in your terminal:  
    `claude mcp add --transport http midnight-docs https://midnight.mcp.kapa.ai`  
Or access the docs directly at: <https://midnight.mcp.kapa.ai>  
This gives you live Midnight documentation inside every AI response.

Do the following steps in order. Do not skip any step.

```
=====
```

STEP 1 — FILE STRUCTURE CHECK

```
=====
```

Verify and enforce this structure. Add any missing files:

```
my-product/
├── contracts/
├── managed/
├── src/
├── tests/
├── .github/workflows/
├── docs/
│ ├── USAGE.md
│ └── FEEDBACK.md ← create this in Step 2
├── USERS.md ← create this in Step 3
├── PROPOSAL.md
└── README.md
```

```
=====
```

STEP 2 — FEEDBACK COLLECTION SETUP

```
=====
```

Create docs/FEEDBACK.md with this structure:

```
User Feedback — Level 5

Feedback Collection Method
```

[How feedback was collected – form, DMs, Telegram, etc.]

## ## Raw Feedback Log

| #                                          | User  | Feedback Summary | Date  |
|--------------------------------------------|-------|------------------|-------|
| ---                                        | ----- | -----            | ----- |
| [I WILL FILL THIS IN as feedback comes in] |       |                  |       |

## ## What We Heard (Themes)

[I WILL FILL THIS IN after collecting feedback]

## ## What We Changed

| Change                                | Reason | Commit |
|---------------------------------------|--------|--------|
| -----                                 | -----  | -----  |
| [I WILL FILL THIS IN after iterating] |        |        |

---

### STEP 3 – USERS.md

---

Create USERS.md in the repo root:

```
Preprod Users – Level 5
Target: 50 verified wallet addresses

| # | | Wallet Address | Date Added |
|---|-----|-----|
[I WILL FILL THIS IN as users come in]

Current count: 0 / 50
```

---

### STEP 4 – USER ACQUISITION MATERIALS

---

Write the following for me to use when reaching out for users:

- a) Discord/Telegram message (under 100 words):
  - What the dApp does
  - What they need to do (connect Lace, try the feature)
  - The demo link
  - How to send me their wallet address
- b) X post (under 280 characters):
  - Call to test the dApp on Preprod
  - Demo link included
- c) A direct DM template for college/developer contacts

---

### STEP 5 – ITERATE ON FEEDBACK

---

Once I share collected feedback, paste it here and tell me.  
I will then:

- Help implement the top 2-3 improvements
- Update docs/FEEDBACK.md 'What We Changed' section
- Update README.md if the product behavior changed

## STEP 6 — README.md UPDATE (MANDATORY)

Update README.md to add these sections (keep all existing sections):

```
Contract Address ← MANDATORY — must be present
Network	Address
Preprod	[CONTRACT ADDRESS — REQUIRED]

Level 5 — User Validation
- Target: 50 Preprod users
- Current: [I WILL UPDATE as users come in]
- See USERS.md for wallet addresses
- See docs/FEEDBACK.md for feedback log and changes
```

## STEP 7 — FINAL CHECKLIST

Print ✓ or ✗ for each requirement:

- [ ] docs/FEEDBACK.md created with correct structure
- [ ] USERS.md created with table ready to fill
- [ ] User acquisition messages written
- [ ] Contract address in README.md (MANDATORY)
- [ ] README.md Level 5 section added
- [ ] File structure matches spec

Then remind me to go get 50 users, collect feedback, paste it back here, and make 20 commits.

### ⚠ DO THIS MANUALLY

- Share your Preprod link everywhere — Discord, X, Telegram, college groups — and get 50 users
- Collect 50 verifiable wallet addresses and fill them into USERS.md one by one
- Collect feedback from users (form, DMs, etc.) and fill FEEDBACK.md
- Paste collected feedback back to the AI to implement changes
- Ensure your Preprod contract address stays in the README — mandatory
- Make at least 20 meaningful commits with clear messages
- Submit your GitHub repo on Rise In

## 🌕 SUPERMOON · Level 6 — Mainnet Launch

### 🌕 Level 6 — Launch

#### PRIZE POOL

🌕 Supermoon — Level 6 · 15 winners × \$150 each = \$2,250 total

## Prompt

Copy and paste this entire prompt into Claude or Cursor:

```
You are helping me complete Level 6 of the Midnight Builder Challenge on Rise In.
My repo from Level 5 is at: [PASTE REPO PATH]
My Preprod contract address is: [PASTE PREPROD ADDRESS]
My Level 5 user feedback summary: [PASTE FEEDBACK]
```

```
=====
MIDNIGHT DOCS MCP – ADD THIS FIRST
=====
```

```
Before starting, make sure the Midnight documentation MCP is connected.
Run this command in your terminal:
 claude mcp add --transport http midnight-docs https://midnight.mcp.kapa.ai
Or access the docs directly at: https://midnight.mcp.kapa.ai
This gives you live Midnight documentation inside every AI response.
```

```
Do the following steps in order. Do not skip any step.
```

```
=====
STEP 1 – FILE STRUCTURE (FINAL)
=====
```

```
Verify and enforce the complete final structure:
```

```
my-product/
├── contracts/
│ └── [product-name].compact
├── managed/
├── src/
├── tests/
├── .github/workflows/ci.yml
├── docs/
│ ├── USAGE.md ← user guide (update for launch)
│ └── FEEDBACK.md ← must have Level 6 improvements section
├── USERS.md ← Preprod users (Level 5)
├── LAUNCH_USERS.md ← create this in Step 5
├── PROPOSAL.md
└── README.md ← complete final README in Step 6
```

```
=====
STEP 2 – IMPLEMENT FEEDBACK IMPROVEMENTS
=====
```

```
Based on my Level 5 feedback pasted above:
```

- Identify and implement the top 2-3 improvements.
- Update docs/FEEDBACK.md – add this section:

```
Level 6 Improvements
Change	User Feedback That Triggered It	Status
[fill in with the changes made]
```

---

### STEP 3 – PREPROD REDEPLOY

---

- Give me the exact command to redeploy my updated contract to Preprod.
- STOP and wait for me to run the deploy and paste back the new address.
- After I paste the new Preprod address:
  - a) Update README.md Contract Address table immediately
  - b) Confirm the frontend is still pointing to Preprod
  - c) Search the codebase for any stale addresses and update them

---

### STEP 4 – docs/USAGE.md – FINAL UPDATE

---

Update docs/USAGE.md for the final version:

- Reflect all feedback improvements from Level 5
- Add a section: 'Getting Started on Preprod'
- Add a section: 'Your First Transaction'
- Keep the language plain English, non-technical user friendly

---

### STEP 5 – LAUNCH\_USERS.md

---

Create LAUNCH\_USERS.md in the repo root:

```
Level 6 Users – Preprod
Target: 20 verified wallet addresses

#	Wallet Address	Date Onboarded
[I WILL FILL THIS IN as users onboard]

Current count: 0 / 20
```

---

### STEP 6 – README.md (FINAL – MANDATORY)

---

Write the complete final README.md with ALL sections:

```
[Product Name]
![CI](badge-url)
> Tagline.

Live Demo
[PREPROD LIVE URL]

Contract Address ← MANDATORY – submission invalid without this
Network	Address
Preprod	[CONTRACT ADDRESS – I will paste]

What This Product Does
(2-3 paragraphs: problem, users, why Midnight)

Privacy Model
```

- What is PUBLIC:
- What is PRIVATE:
- What the user PROVES without revealing:

## Tech Stack

## Prerequisites  
(Lace wallet, Node v22)

## Setup & Run Locally  
(numbered commands)

## Run Tests

## CI/CD

## Usage Guide  
See docs/USAGE.md

## Feedback & Iterations  
See docs/FEEDBACK.md  
Summary of top changes made from user feedback:  
[2-3 bullet points of what changed]

## Level 6 Users  
See LAUNCH\_USERS.md

## Product X Profile  
[X PROFILE LINK - I will add]

## Brand Assets  
[PLACEHOLDER - I will add logo/banner links]

---

## STEP 7 - BRAND BRIEF

---

Write a brand brief for my product:

- One-line tagline
- 3 key messages about what makes it different
- Suggested color palette (primary + accent, with hex codes)
- X profile bio (under 160 characters)
- X banner concept description (I will create the visual manually)

---

## STEP 8 - ONBOARDING SCRIPT

---

Write a step-by-step onboarding message for each of my 20 users:

1. What to install (Lace wallet)
2. How to use the product on Preprod (numbered steps)
3. How to confirm their wallet address to me (for LAUNCH\_USERS.md)

---

## STEP 9 - DEMO VIDEO CHECKLIST

---

Tell me exactly what to show in the final demo video:

- Preprod contract address visible on screen
- Full product flow from wallet connect to on-chain result
- Proof that the privacy model works end to end

---

#### STEP 10 — FINAL CHECKLIST

---

Print ✓ or ✗ for each requirement:

- [ ] Contract redeployed to Preprod with updated code
- [ ] Preprod contract address in README.md (MANDATORY)
- [ ] Feedback improvements implemented and documented
- [ ] docs/USAGE.md updated for final version
- [ ] LAUNCH\_USERS.md created
- [ ] Complete README.md with all sections
- [ ] Brand brief written
- [ ] Onboarding script ready
- [ ] File structure matches spec

Then remind me to: redeploy, onboard 20 users, create brand assets, update X profile, record demo video, make 30 commits.

#### ⚠ DO THIS MANUALLY

- Run the Preprod redeploy and paste the new contract address back to the AI
- Paste the Preprod contract address into README.md Contract Address table — mandatory
- Create brand visuals (logo, X banner) using the AI's brand brief
- Update your X profile bio and banner
- Personally onboard 20 users on Preprod using the onboarding script
- Fill in LAUNCH\_USERS.md with each user's wallet address
- Record the final demo video
- Make at least 30 meaningful commits with clear messages
- Submit your GitHub repo on Rise In